

Milestone 1 Documentation: Advanced System Engineering and Personalization

Project Title: Developing an AI System for Personalized Content
Creation in Media

Intern Name: Naina Anil Pandey

Internship Program: Infosys Springboard 6.0

Milestone 2 Period: February 7 – March 2, 2026

1. Introduction

Following the successful exploration of multiple Large Language Models (LLMs) and API integrations in Milestone 1, the objective of Milestone 2 shifted towards robust backend engineering. The focus was on building an intelligent, self-correcting backend system capable of hyper-personalized content generation. By utilizing **Gemini 2.5 Flash** as the primary cognitive engine in my project, the system now dynamically adapts to complex user parameters, enforces content-specific validation rules, analyzes text quality using NLP metrics, and seamlessly manages user data (profiles and version history) completely via the backend infrastructure.

2. Core Backend Architecture & The Gemini Engine

The system's backend is powered by a custom *LangChain* pipeline, utilizing Google's Gemini 2.5 Flash (*ChatGoogleGenerativeAI*) as the primary model. *Gemini* was selected for its high inference speed and strict adherence to complex, multi-constraint prompt templates.

High-Availability Fallback Mechanism

To ensure zero downtime, the backend features a robust "Try-Except" cascade. If *Gemini* faces rate limits or network issues, the system automatically redirects the exact prompt to *Groq* (Llama 3.3), and if that fails, it falls back to a locally hosted *Ollama* (Llama 3.2:1B) model. This guarantees the user always receives content regardless of API status.

Dynamic Prompt Engineering Matrix

The backend dynamically compiles user inputs into a structured *ChatPromptTemplate* before sending it to *Gemini*.

- **The Prompt Structure:** The system injects over 12 variables (Brand, Goal, Audience, Persona, Tone, Format, Length, Keywords, Words to Avoid, Emojis, Hashtags, CTA) into a strict instruction set.
- **Temperature / Creativity:** The backend maps a user-defined "Creativity Slider" directly to Gemini's temperature parameter (ranging from 0.0 to 1.0). This allows the system to generate highly factual content (low temperature) or highly creative variations (high temperature) simultaneously during "Compare Mode."

3. Impact of Dynamic Inputs on Content Generation

One of the most significant backend achievements is how Gemini reacts to missing versus provided dynamic parameters.

A. When Specific Inputs are Provided

If a user inputs the topic *"Launching a new software tool"* and specifies the tone as **"Witty,"** the audience as **"Gen Z Consumers,"** and adds a negative prompt **"Avoid: cheap, guaranteed,"** Gemini drastically alters its neural path. It bypasses corporate jargon, adopts a conversational voice, and completely suppresses the restricted vocabulary.

B. When Inputs are Omitted (The "Smart Defaults" Engine)

If a user only provides the raw topic and selects the Content Type, the backend does not fail or generate generic text. Instead, a backend Python logic tree kicks in to assign **Smart Defaults**.

- *Example:* If the user selects "LinkedIn Post" but omits parameters, the backend silently injects: *Audience = Corporate Executives, Persona = Industry Thought Leader, Tone = Engaging, Format = Bullet Points.*

This ensures Gemini always receives a fully optimized prompt matrix, even from a minimal user request.

4. Multi-Content Type Formatting & Validation

To demonstrate Gemini's contextual awareness, the backend routes the exact same user topic into entirely different content structures based solely on the designated *content_type*.

The Identical Input Test:

- **Topic:** *"We are integrating AI into our media platform."*
- **Output 1 - Professional Email:** Gemini structures the output with formal salutations, precise paragraph breaks, and zero emojis.
- **Output 2 - LinkedIn Post:** Gemini pivots to an engaging hook, bullet points, abundant emojis, and industry hashtags.
- **Output 3 - Ad Copy:** Gemini produces a short, punchy, persuasive snippet focused on ROI and a strong Call to Action.

Post-Generation Validation Logic:

LLMs can occasionally hallucinate or ignore formatting constraints. To counter this, a custom backend *validate_content()* function processes Gemini's output *after* generation:

1. **Hashtag Injection:** If the user requested hashtags but Gemini missed them, the backend automatically appends contextual tags (e.g., #Trending #Innovation).
2. **CTA Verification:** The backend scans the output array for action verbs ("click", "subscribe", "reply"). If missing, it forcefully appends the user's requested CTA.
3. **Subject Line Enforcement:** For emails, if the string lacks "Subject:", the backend prepends a relevant subject line. The system then flags these corrections (e.g., " ⚠️ Fixed: Added Missing CTA") so the user knows the backend intervened.

5. Content Quality Analysis (NLP Module)

Rather than relying solely on the LLM's output, a custom Natural Language Processing (NLP) module was built into the backend (`analyze_text()`) to grade the generated text mathematically:

- **Sentiment Analysis:** The backend scans the generated text array against internal dictionaries of positive/negative lexicon to categorize the output as *Positive*, *Negative*, or *Neutral*.
- **Readability Scoring:** By calculating the Words-Per-Sentence (WPS) ratio, the backend proxy-scores the text as *Easy*, *Standard*, or *Advanced* reading levels.
- **Keyword Density:** The script iterates through user-defined target keywords to mathematically verify and count their inclusion in the final string.

6. Data Persistence: User Profiles and Version History

To elevate the application to enterprise-level utility, backend memory management was implemented using local JSON storage mechanisms.

- **Profile Preferences (*prefs.json*):** The backend actively writes and reads user baseline preferences (Default Brand, Tone, Persona). When initialized, the backend fetches this JSON and pre-populates the LangChain input dictionary, saving user time.
- **Version Management (*history.json*):** Every successful generation run (along with its metadata, timestamp, engine used, and metrics) is appended to a backend JSON database. This allows the system to recall previous content states & serve them back to the user.

7. Conclusion

In Milestone 2, the application evolved from a simple API wrapper into a highly controlled AI Content Engine. By leveraging advanced Prompt Engineering on the Gemini 2.5 Flash model, implementing rigorous Python-based validation algorithms, and integrating local data persistence, the backend now guarantees output that is consistently formatted, contextually aware, and mathematically verified for quality.